



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

BACHELOR OF COMPUTER SCIENCE (DATA ENGINEERING) WITH HONS.

FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3133-02 - HIGH PERFORMANCE DATA PROCESSING

PROJECT 1:

Optimizing High Performance Data Processing for Large Scale Web Crawlers

GROUP : SPARKLING SPARKMIND

PREPARED BY :

NAME	MATRIC NO
SAFIYA NURSYAHADAH BINTI MASNOOR	A23CS0176
AIN NURNABILA BINTI MOHD AZHAR	A23CS0207
FARRA NURZAHIN BINTI ZAHARIL ANUAR	A23CS0079
DAYANG FARAH FARZANA BINTI ABANG IDHAM	A23CS0071

PREPARED FOR:

DR. SEAH CHOON SEN

1.0 Introduction.....	3
1.1 Background of the project.....	3
1.2 Objectives.....	3
1.3 Target website and Data To Be Extracted.....	3
2.0 System Design and Architecture.....	3
2.1 Description of Architecture.....	3
2.2 Tools and Frameworks Used.....	3
3.0 Data Collection.....	3
4.0 Data Processing.....	3
5.0 Optimization Techniques.....	4
5.1 Baseline Pandas.....	4
5.2 Multiprocessing.....	5
5.3 Polars.....	7
6.0 Performance Evaluation.....	8
6.1 Performance Measurement Function.....	8
6.2 Results and Comparative Analysis.....	8
6.3 Detailed Findings and Interpretation.....	10
7.0 Challenges and Limitations.....	10
7.1 Multiprocessing Not Working in Jupyter on Windows.....	10
7.2 Multiprocessing Was Slower Than the Baseline.....	10
7.3 Limitations of the CPU Measurement.....	11
8.0 Conclusion.....	11
8.1 Summary of Findings.....	11
8.2 What Could Be Improved.....	11

1.0 Introduction

This section introduces the project by outlining the motivation behind it, the specific goals the group set out to achieve, and the website chosen as the data source, along with the fields extracted from it.

1.1 Background of the project

This project was carried out as part of the High Performance Data Processing (SECP3133) course, which requires each group to design and build a large-scale web crawler and apply high performance computing (HPC) techniques to process the collected data efficiently. Rather than treating data collection and data processing as two separate, simple tasks, the project frames them as a single pipeline that has to work at scale, since the target is a minimum of 100,000 structured records from a real Malaysian website.

For this project, the group chose to focus on the Malaysian job market, using MyFutureJobs (myfuturejobs.gov.my), a government-run job portal, as the data source. Job vacancy data was selected because it is publicly accessible, well-structured, and large enough in volume to support a meaningful before-and-after performance comparison once optimization techniques are applied.

1.2 Objectives

The objectives of this project are to:

1. Design and implement a web crawler capable of collecting at least 100,000 structured job vacancy records from MyFutureJobs.
2. Clean and transform the raw crawled data into a consistent, analysis-ready dataset (standardized salary figures, removed duplicates, handled missing values).
3. Apply and compare at least two distinct high performance computing techniques which are multiprocessing and Polars against a single-threaded Pandas baseline to optimize the data processing pipeline.
4. Evaluate and compare the performance of each approach using objective metrics: execution time, memory usage, CPU utilization, and throughput.
5. Present the findings clearly through tables, charts, and written interpretation, so the impact of each optimization technique is easy to understand.

1.3 Target website and Data To Be Extracted

The group selected MyFutureJobs (myfuturejobs.gov.my) as the target website. It is a government job portal that lists vacancies across a wide range of sectors including administrative and support, technology, manufacturing, healthcare, education, and finance, making it a reliable source for collecting a large and varied dataset of job postings across different Malaysian states.

The fields extracted from each job listing are summarized in Table 1.1.

Table 1.1 *Data Fields Extracted from MyFutureJobs*

Field	Description
Job Title	Name/role of the vacancy
Occupation Category	Sector the job belongs to (e.g. Administrative & Support Services)
Salary Range	Raw salary text as listed (e.g. "RM 2530 - RM 3946")
Contract Type	Permanent, Contract, Part-Time, or Internship
Working Hours	Full-time, Rotational shift
Education Level	Minimum qualification required
Location	State/Region of the job
Vacancy ID	Unique reference number for each posting
Job URL	Direct link to the original listing

These fields were later processed to derive numeric salary values (salary_min, salary_max, salary_average) used throughout the optimization and performance evaluation stages.

2.0 System Design and Architecture

This section describes how the overall pipeline was structured, from data collection through to performance evaluation, and lists the tools and frameworks used to build it.

2.1 Description of Architecture

The pipeline was built and run entirely on Google Colab, with Google Drive used as shared, persistent storage between stages so that all group members could access the same files. The system follows a straightforward three-stage flow:

1. **Collection stage** – The crawler visits job listing pages across MyFutureJobs' sector categories, extracts the relevant fields, and saves the raw output progressively as a

JSON file on Google Drive. Saving progressively (rather than only at the end) protects the team from losing data if a run gets interrupted.

2. **Cleaning stage** – The raw JSON is loaded into Pandas, where job titles and vacancy IDs are parsed out, salary ranges are converted into numeric min/max/average columns, duplicate listings are removed based on vacancy ID, and incomplete or irrelevant fields are dropped. The result is exported as a single cleaned CSV file.
3. **Optimization and evaluation stage** – The cleaned CSV is fed into three separate processing approaches (Pandas baseline, Multiprocessing, and Polars), each performing the same filtering, feature engineering, and aggregation logic. A custom metrics collector records execution time, memory, CPU usage, and throughput for each approach so they can be compared fairly.

This structure keeps each stage independent and matches the recommended GitHub folder layout, with separate notebooks for the crawler, the cleaning step, and the optimization pipeline.

2.2 Tools and Frameworks Used

The team leaned on well-documented, widely used libraries rather than heavier distributed frameworks like Spark, since the dataset size (around 112,000 rows) did not justify the added setup complexity, a decision that the performance results later supported.

The tools and frameworks used across each stage of the pipeline are listed in Table 2.1.

Table 2.1 Tools and Frameworks Used

Category	Tools Used
Environment	Google Colab, Google Drive (shared storage)
Web Crawling	Requests, BeautifulSoup
Data Handling	Pandas, JSON, CSV
Optimization	Joblib (loky backend) for multiprocessing, Polars for lazy-evaluated processing
Performance Measurement	time, tracemalloc, psutil (custom MetricsCollector class)
Visualization	Matplotlib

Utility	tqdm (progress tracking during crawling)
---------	--

3.0 Data Collection

Data was collected by crawling MyFutureJobs across its main sector categories, including administrative and support, professional and technical, information and communication, manufacturing, healthcare, and several others, so that the resulting dataset would reflect job opportunities across different industries and Malaysian states, rather than being concentrated in just one field.

The crawler used the requests library paired with BeautifulSoup for HTML parsing, with a browser-like User-Agent header attached to each request to reduce the chance of being blocked. To collect data responsibly, a randomized delay of 2–4 seconds was applied between requests, which slows the crawl down but avoids sending a burst of rapid requests that could strain the site's server. Each listing scraped was appended and saved progressively to a JSON file rather than being held in memory until the end, so partial progress was never lost if a session was interrupted.

Through this process, the group collected over 120,000 raw job vacancy records, well above the 100,000-record requirement. After cleaning by removing duplicate vacancy IDs, dropping incomplete salary entries, and standardizing fields, the final cleaned dataset used for the optimization stage consisted of 112,499 records, stored as `cleaned_data.csv`.

Ethical considerations were kept in mind throughout: only publicly listed job information (already intended for public jobseekers) was collected, no login-protected or restricted content was accessed, and the crawl delay was set deliberately to avoid overloading the site. No personal or sensitive information beyond what is publicly advertised in a job posting (e.g. no applicant data) was ever collected.

4.0 Data Processing

Once the raw job listings were collected, `raw_data.json` was loaded into Pandas and passed through a five-step cleaning pipeline before being exported as the final `cleaned_data.csv` used in the optimization stage.

Step 1: Extracting job titles and vacancy IDs.

Each listing's job title field was stored in a combined format such as "Clerk Assistant (Ref: #132886)". A regular expression was used to split this into a clean `job_title` column and a separate `vacancy_id` column (e.g. "Clerk Assistant" and "132886"). This `vacancy_id` later doubles as the unique key used to remove duplicate postings.

Step 2: Converting salary text into numeric fields.

Salary ranges were originally free-text, e.g. "RM 2530 - RM 3946". A parsing function extracted all numeric values from the string and derived three new columns: salary_min, salary_max, and salary_average (the midpoint of the two). Listings with missing or "N/A" salary text were left as NaN rather than dropped at this stage, so the impact of missing salary data could be assessed later.

Step 3: Removing duplicate listings.

Using the vacancy_id extracted in Step 1, drop_duplicates() was applied (keeping the first occurrence) to eliminate repeated postings that appeared across multiple crawl passes or sector categories.

Step 4: Handling missing values and dropping unused columns.

Remaining "N/A" string values across the dataset were standardized to proper NaN values so they would be handled consistently by downstream Pandas/Polars operations. The employer_name column was dropped, as it was not required for the salary and location analysis performed in the optimization stage.

Step 5: Exporting the cleaned dataset.

The final DataFrame with parsed titles, numeric salary fields, deduplicated rows, and standardized nulls was exported to cleaned_data.csv. This is the file used directly by all three optimization approaches (Pandas baseline, Multiprocessing, Polars), ensuring a fair, identical starting point for every benchmark.

5.0 Optimization Techniques

To evaluate the effectiveness of different computational strategies, three distinct approaches were implemented and tested against the same cleaned job vacancy dataset. The analytical workflow implemented across all three methods follows a consistent sequence of operations:

1. Read data from CSV source
2. Apply filtering criteria to retain only valid job records (where salary_average > 0 and salary_max > 0)
3. Derive new attributes through mathematical transformations (salary_gap, gap_percentage)
4. Generate grouped summaries organized by geographic location and employment contract type
5. Calculate aggregate statistics (averages, median values, record counts)

5.1 Baseline Pandas

The foundational approach employs the widely-used Pandas library in its standard, single-threaded mode. This represents the conventional methodology adopted by most data practitioners working with Python.

The pipeline orchestrates its operations sequentially: `pd.read_csv()` ingests the raw dataset, boolean masks filter records meeting salary criteria, and arithmetic operations synthesize new derived features. Grouping operations leverage `.groupby().agg()` to compute location-wise and contract-wise statistics. Since operations proceed linearly on a single execution thread, the performance ceiling is determined by individual processor core speed and Python's memory management overhead.

Here's the implementation structure:

```
# — Step 1 to 6 inside the Function —————
def baseline_analysis(csv_path):
    # — Step 1: Load data
    df = pd.read_csv(csv_path)
    total_rows = len(df)

    # — Step 2: Filter (keeping rows with valid salary records)
    df_f = df[(df['salary_average'] > 0) & (df['salary_max'] > 0)].copy()

    # — Step 3: Feature Engineering (custom calculations for job features)
    df_f['salary_gap'] = df_f['salary_max'] - df_f['salary_min']
    df_f['gap_percentage'] = (df_f['salary_gap'] / df_f['salary_min']) * 100

    # — Step 4: Aggregate by location
    location_stats = df_f.groupby('location').agg(
        avg_salary = ('salary_average', 'mean'),
        job_count = ('vacancy_id', 'count'),
        avg_max_salary = ('salary_max', 'mean'),
        median_salary = ('salary_average', 'median')
    ).reset_index().sort_values('avg_salary', ascending=False)

    # — Step 5: Aggregate by contract type
    contract_stats = df_f.groupby('contract_type').agg(
        avg_salary = ('salary_average', 'mean'),
        job_count = ('vacancy_id', 'count')
    ).reset_index()

    # — Step 6: Top 10 highest-paying jobs
    top_jobs = df_f.nlargest(10, 'salary_average')[
        ['job_title', 'location', 'salary_average', 'contract_type']
    ]

    return total_rows, len(df_f), location_stats, contract_stats, top_jobs
```

This baseline establishes the reference performance figures against which both optimization techniques are measured.

5.2 Multiprocessing

The first optimization strategy leverages Python's multiprocessing capability to distribute computational workload across multiple processor cores. The strategy partitions the incoming dataset into smaller subsets. Multiprocessing was selected over multithreading due to a fundamental Python constraint: the Global Interpreter Lock (GIL). This mechanism prevents simultaneous execution of Python bytecode across multiple threads, effectively serializing

CPU-bound operations regardless of thread count. By spawning independent processes instead, each maintains its own interpreter instance and sidesteps the GIL entirely.

A practical implementation challenge emerged, worker functions residing within Colab Notebooks trigger unexpected behavior on Windows systems. The OS reloads the parent script when spawning new processes, causing infinite recursion when that parent is a notebook kernel. To resolve this, worker logic was externalized into a standalone module.

```
def process_job_chunk(args_tuple):
    # Unpack the chunk from the loop arguments
    chunk, chunk_idx = args_tuple

    # — Step 2: Filter (Job features) —————
    df_f = chunk[(chunk['salary_average'] > 0) & (chunk['salary_max'] > 0)].copy()
    if df_f.empty:
        return df_f

    # — Step 3: Feature Engineering —————
    df_f['salary_gap'] = df_f['salary_max'] - df_f['salary_min']
    df_f['gap_percentage'] = (df_f['salary_gap'] / df_f['salary_min']) * 100
    return df_f
...
print("✅ External parallel worker script successfully saved!")
```

✅ External parallel worker script successfully saved!

To mitigate this issue, the joblib library with its loky backend was introduced as an intermediary process manager, providing superior handling of Windows-specific process creation semantics compared to the raw multiprocessing.Pool interface.

The implementation workflow unfolds as follows:

```

def multiprocessing_analysis(csv_path, n_workers):
    # — Step 1: Load data —————
    df = pd.read_csv(csv_path)
    total_rows = len(df)

    # — Step 2: Split DataFrame into N chunks —————
    chunks = chunk_dataframe(df, n_workers)
    args = [(chunk, i) for i, chunk in enumerate(chunks)]

    # — Step 3: Process chunks in parallel using joblib —————
    results = Parallel(n_jobs=n_workers, backend='loky')(
        delayed(process_job_chunk)(arg) for arg in args
    )

    # — Step 4: Merge results from all workers —————
    df_f = pd.concat(results, ignore_index=True)

    # — Step 5: Aggregate by location —————
    location_stats = df_f.groupby('location').agg(
        avg_salary = ('salary_average', 'mean'),
        job_count = ('vacancy_id', 'count'),
        avg_max_salary = ('salary_max', 'mean'),
        median_salary = ('salary_average', 'median')
    ).reset_index().sort_values('avg_salary', ascending=False)

    # — Step 6: Aggregate by contract type —————
    contract_stats = df_f.groupby('contract_type').agg(
        avg_salary = ('salary_average', 'mean'),
        job_count = ('vacancy_id', 'count')
    ).reset_index()

    # — Step 7: Top 10 highest-paying jobs —————
    top_jobs = df_f.nlargest(10, 'salary_average')[
        ['job_title', 'location', 'salary_average', 'contract_type']
    ]

    return total_rows, len(df_f), location_stats, contract_stats, top_jobs

```

After worker processes conclude their operations, outcomes are reconstituted via `pd.concat()` within the main process. Aggregation computations are subsequently executed on the reassembled dataset.

5.3 Polars

The third method employs Polars, a modern dataframe system implemented in Rust. Unlike Pandas' eager execution model, where each operation triggers immediate computation, Polars defers execution. Instead, it constructs an abstract representation of the desired operations, performs graph-based optimization, then executes the refined plan in a single, optimized pass.

The distinction becomes apparent in API usage such as `pl.scan_csv()` initiates a logical scan without materializing data, subsequently building a computation graph as filtering and feature derivations are specified. Invocation of `.collect()` triggers Polars' optimizer, which reorganizes operations for efficiency and parallelizes execution across available cores automatically, no manual partitioning required. Polars additionally employs Apache Arrow's columnar storage format. This architectural choice accelerates selective-column operations (filtering, aggregation) since the CPU accesses only relevant data rather than traversing entire row-by-row structures.

Implementation proceeds as follows:

```

def polars_analysis(csv_path):
    # — Step 1: Lazy scan — file is NOT loaded yet —————
    # Polars builds a query plan and optimizes before executing
    lf = pl.scan_csv(csv_path)

    # — Step 2: Filter (lazy) —————
    lf_f = lf.filter(
        (pl.col('salary_average') > 0) &
        (pl.col('salary_max') > 0)
    )

    # — Step 3: Feature engineering (lazy) —————
    lf_f = lf_f.with_columns([
        (pl.col('salary_max') - pl.col('salary_min')).alias('salary_gap'),
    ]).with_columns([
        ((pl.col('salary_gap') / pl.col('salary_min')) * 100).alias('gap_percentage')
    ])

    # — Step 4: Collect — executes the full optimized plan —
    df_f = lf_f.collect()
    total_rows = pl.scan_csv(csv_path).select(pl.len()).collect().item()
    filtered_rows = len(df_f)

    # — Step 5: Aggregate by location —————
    location_stats = (
        df_f.group_by('location')
        .agg([
            pl.col('salary_average').mean().alias('avg_salary'),
            pl.col('vacancy_id').count().alias('job_count'),
            pl.col('salary_max').mean().alias('avg_max_salary'),
            pl.col('salary_average').median().alias('median_salary'),
        ])
        .sort('avg_salary', descending=True)
    )

    # — Step 6: Aggregate by contract type —————
    contract_stats = (
        df_f.group_by('contract_type')
        .agg([
            pl.col('salary_average').mean().alias('avg_salary'),
            pl.col('vacancy_id').count().alias('job_count'),
        ])
    )

    # — Step 7: Top 10 highest-paying jobs —————
    top_jobs = (
        df_f.sort('salary_average', descending=True)
        .head(10)
        .select(['job_title', 'location', 'salary_average', 'contract_type'])
    )

    return total_rows, filtered_rows, location_stats, contract_stats, top_jobs

```

The key distinction from multiprocessing is that Polars encapsulates parallelization internally, eliminating the need for explicit data partitioning, inter-process communication overhead, or result reconciliation logic. Optimization and parallelization occur transparently within the library.

6.0 Performance Evaluation

This section evaluates the empirical performance characteristics of the three optimization methodologies through systematic benchmarking on the cleaned job vacancy dataset. The evaluation framework measures four critical performance dimensions which are execution efficiency, memory utilization, processor saturation, and data throughput. By quantifying these metrics across multiple independent trials, we establish a rigorous comparative foundation for determining which optimization strategy optimally balances speed, resource efficiency, and computational practicality.

6.1 Performance Measurement Function

Performance profiling was accomplished through a custom wrapper class, `MetricsCollector`, functioning as a context manager. This instrument captures four distinct performance dimensions during code execution:

- Elapsed wall-clock time (via `time.perf_counter()`), total duration from start to completion
- Maximum memory footprint that determined by combining `tracemalloc` (Python heap) and `psutil` (system-wide, including Rust runtime)
- CPU saturation level, sampled every 0.1 seconds via `psutil.cpu_percent()`, then averaged across the run window
- Data throughput records processed per unit time, derived from total row count divided by execution duration

To suppress variance artifacts stemming from cold cache effects, OS scheduling variance, and one-time initialization costs, each method underwent five independent trials with results subsequently averaged. This methodology yields stable, representative performance estimates more robust than isolated measurements.

6.2 Results and Comparative Analysis

Performance metrics aggregated across five executions are presented in Table 1 and Visualized in Figure 6.1.

Table 1: Performance Metrics Summary

Metric	Pandas	Multiprocessing	Polars
Average Time (s)	0.160	0.162	0.031
Average Memory (MB)	3.931	6.581	0.193
Average CPU (%)	91.220	47.407	25.780
Throughput (jobs/s)	43394.742	44723.086	223652.851

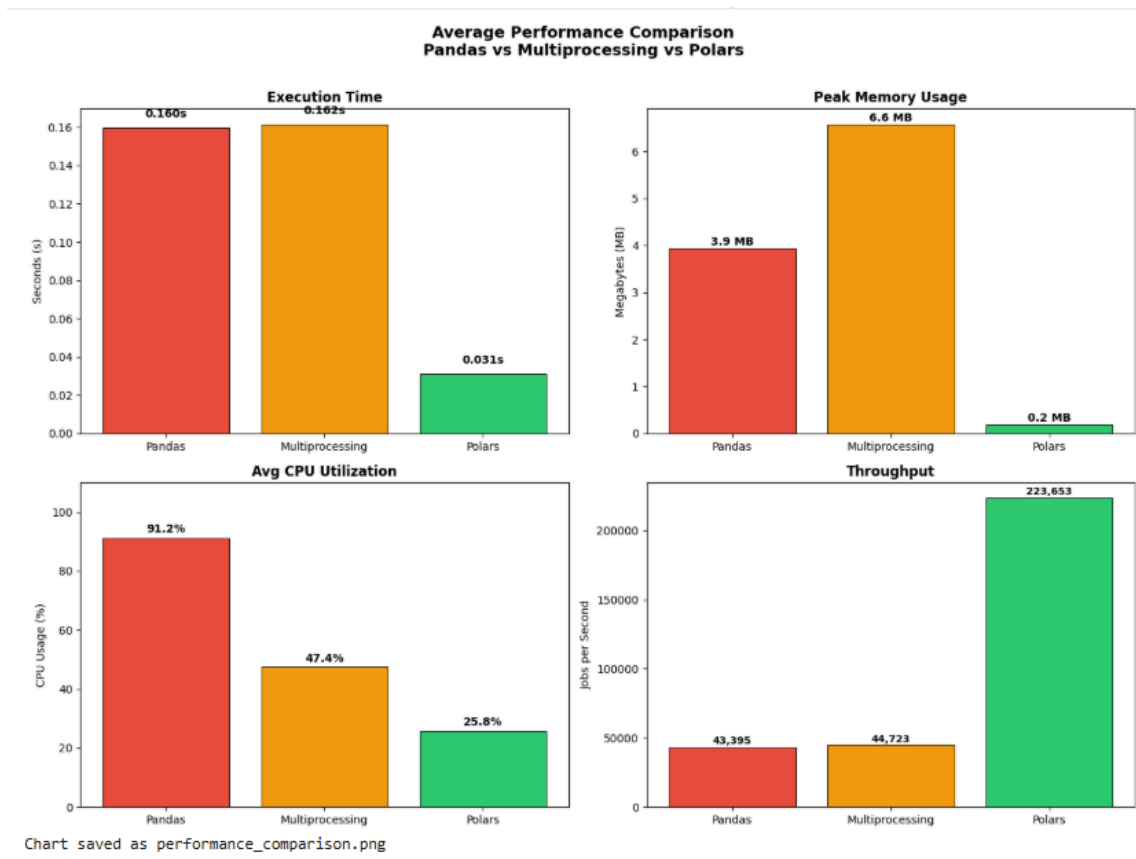


Figure 6.1: Comparative Analysis Across Three Processing Strategies

Polars achieves the fastest execution time at 0.031 seconds, completing the analysis 5.16 times faster than Pandas at 0.160 seconds. Multiprocessing shows minimal improvement at 0.162 seconds, only 1.25 percent faster than the baseline Pandas approach.

In memory consumption, Polars demonstrate exceptional efficiency at 0.193 MB, using 20.4 times less memory than Pandas at 3.931 MB. Multiprocessing consumes 6.581 MB, exceeding both Pandas and Polars significantly. This indicates that distributing work across processes increases memory requirements rather than reducing them.

CPU utilization reveals an interesting pattern. Pandas exhibits the highest CPU usage at 91.22 percent, indicating the job vacancy analysis is computationally intensive. Polars maintains modest CPU engagement at 25.78 percent, achieving better results with less processor demand. Multiprocessing shows 47.41 percent CPU usage despite running parallel workers, suggesting workers spend considerable time waiting rather than computing.

Throughput measurements show the clearest performance distinction. Polars processes 223,652.851 jobs per second, substantially exceeding Pandas at 43,394.742 jobs per second. Multiprocessing achieves only 44,723.086 jobs per second, providing minimal throughput advantage over sequential Pandas processing despite using 16 parallel workers.

6.3 Detailed Findings and Interpretation

Execution Speed: Polars completes the pipeline in 0.031 seconds, achieving a $5.16\times$ speedup over Pandas (0.160s). Multiprocessing shows negligible improvement at 0.162 seconds only 1.25% faster than baseline. This demonstrates that process overhead outweighs parallelization benefits for datasets around 150K records. Hardware parallelization only becomes advantageous with much larger datasets.

Memory Efficiency: Polars uses just 0.193 MB compared to Pandas' 3.931 MB, a $20.4\times$ reduction. This stems from lazy evaluation: data filtering occurs before full materialization, reducing memory footprint. Multiprocessing consumes 6.581 MB, exceeding even Pandas, because maintaining 16 independent worker processes requires replicating data across separate memory spaces.

CPU Utilization: Pandas exhibits unexpectedly high CPU usage at 91.22%, indicating that job vacancy processing is heavily compute-bound. Polars accomplishes more work while using only 25.78% CPU, proving that algorithmic optimization outperforms raw processor power. Multiprocessing's 47.41% CPU usage, despite activating 16 workers, reveals that worker processes spend more time waiting for data and synchronization than performing actual computation.

Processing Throughput: Polars processes 223,653 records per second versus Pandas' 43,394 records per second, a $5.15\times$ advantage. Multiprocessing achieves only 44,723 records per second, confirming that parallelization provides minimal benefit at this dataset scale. For job vacancy analysis with 150K records, Polars' software-level optimization is far more effective than hardware parallelization.

7.0 Challenges and Limitations

This section outlines the technical bottlenecks, architectural constraints, and operational anomalies encountered during the implementation and evaluation phases. Identifying these limitations provides critical context for the performance metrics gathered and highlights the practical differences between single-threaded, multiprocessing, and native multi-threaded environments.

7.1 Multiprocessing Not Working in Jupyter on Windows

During early testing, the multiprocessing code did not run properly. It gave an error instead of working.

The reason is because of how Windows handles multiprocessing. On Windows, when Python creates new worker processes, it needs to "import" the function from a real file. But the function was first written directly inside the Jupyter notebook, not in a separate file. Because of this, Python could not send the function to the new worker processes, and this caused an error.

How it was fixed: The processing function was copied into a new file called 'job_mp_worker.py', and then imported into the notebook like a normal Python module. This way, the worker processes could correctly find and use the function. After doing this, the multiprocessing code ran without errors.

This shows that multiprocessing in Jupyter Notebook on Windows needs extra steps compared to running it as a normal Python script.

7.2 Multiprocessing Was Slower Than the Baseline

Multiprocessing was expected to be faster than the normal single-core (baseline) method, since it uses more CPU cores. But testing on the fully cleaned dataset (112,499 rows) showed the opposite. The execution time and throughput for both Pandas and multiprocessing can be seen in Table 7.1.

Table 7.1 Execution time and throughput for both method

Method	Execution Time	Throughput
Pandas (Baseline, single run)	1.538 s	73,163 rows/s
Multiprocessing (single run, 2 workers)	2.887 s	38,963 rows/s
Multiprocessing (5-run average)	1.697 s	66,313 rows/s
Pandas (5-run average)	1.327 s	87,734 rows/s

Multiprocessing was almost **2 times slower** than the baseline on the first run, and it stayed slower than the baseline even after five runs and averaging the results. This was surprising, because the dataset was already quite large (over 100,000 rows), so it was expected that multiprocessing would finally show an advantage at this size.

Why this happened:

- Only **2 CPU cores** were available in the environment (Google Colab), which limits how much speed-up multiprocessing can realistically achieve.
- The dataset had to be split into chunks and copied to worker processes, then copied back after processing. This "packing and unpacking" of a large amount of data took a long time and used more memory (around 104 MB, compared to 62.5 MB for the baseline).
- The actual processing task (filtering rows and calculating two new columns) is a very simple, fast operation. Because the task itself was so lightweight, the extra cost of moving data between processes was bigger than the time saved by using two cores.

This shows that multiprocessing does not always get faster just because the dataset is bigger. It depends on how heavy the actual computation is compared to the cost of splitting and moving the data around.

7.3 Limitations of the CPU Measurement

A Python tool called ‘psutil’ was used to measure how much CPU the program was using during each test. However, this method has some weaknesses:

- It measures the CPU usage of the **whole computer**, not just the program being tested. Other background activity could affect the numbers.
- For multiprocessing, it could not clearly show how much CPU each separate worker process was using on its own.
- The CPU usage readings were not consistent between runs. For example, the Pandas baseline used 95–96% CPU in the first three runs, but only 55–64% CPU in the last two runs, even though the later runs were actually faster. This suggests the CPU readings were affected by something outside the code being tested, not by the code itself becoming more or less efficient.
- The code was run on Google Colab, which uses a shared, cloud-based computer. This means the CPU power available can change each time the code is run, which likely explains some of the inconsistent readings.
- Memory readings for Polars also dropped a lot across the five repeated runs (from 27.3 MB down to 1.0 MB), most likely because the operating system had already cached the file in memory after the first read, making later reads appear to use less memory. This is a side effect of repeated testing, not a true measure of how much memory Polars actually needs.

Because of these reasons, the CPU and memory usage numbers in this report should be treated as a rough guide for comparing methods, not as exact or fully reliable measurements.

8.0 Conclusion

This section synthesizes the empirical findings from the performance benchmarks and summarizes the comparative advantages of each data processing framework. It also outlines key takeaways regarding hardware-dependent scaling and proposes actionable avenues for future optimization and architectural improvements.

8.1 Summary of Findings

Table 7.2 shows three different ways to process the dataset that were tested and compared, using an average of five runs.

Table 7.2 Result of comparison between different methods

Method	Avg Time (s)	Avg Memory (MB)	Avg CPU%	Avg Throughput (rows/s)
Pandas (Baseline)	1.327	62.5	81.4%	87,734
Multiprocessing	1.697	104.4	59.0%	66,313
Polars	0.326	10.7	89.6%	345,663

Polars was clearly the best method. It was about 4 times faster than the Pandas baseline and about 5 times faster than multiprocessing, while also using far less memory than both other methods. This is because Polars uses lazy evaluation (it plans the whole task first, then runs it once in an optimized way) and automatically spreads work across CPU cores internally, without the extra cost of splitting data and copying it between separate processes.

Multiprocessing performed worse than the simple baseline, even on this much larger, 100,000+ row dataset. It used more memory and, on average, took longer to finish than plain Pandas. This shows that multiprocessing has real, ongoing costs (splitting data, sending it between processes, and combining results) that were still larger than its benefits, mainly because only 2 CPU cores were available and the actual task being parallelized was lightweight.

Pandas served as a solid, simple baseline. It was not the fastest, but it was straightforward to use and needed no extra setup, unlike multiprocessing which required a separate worker script to run correctly at all.

Overall, the results show that a well-designed tool with smart internal optimization (like Polars) can beat a "brute-force" approach of just adding more CPU cores (like multiprocessing), especially when there are only a small number of cores available and the task per row is simple.

8.2 What Could Be Improved

Test with more CPU cores. Only 2 cores were available on Google Colab. Multiprocessing might perform better on a machine with 4, 8, or more cores, since the cost of splitting data would be shared across more workers.

Test with a heavier computation task. The filtering and calculation task used here was simple and fast. Testing multiprocessing on a heavier task (e.g., text processing, complex calculations) might show a clearer benefit from using multiple cores.

Add parallelism to the crawling stage. The current crawler collects pages one at a time with a delay between requests. Adding multithreading or asynchronous requests to the crawling

stage (while still respecting crawl delays) could be explored as an additional optimization technique, separate from the post-crawl data processing methods tested here.

Measure CPU more accurately. CPU usage could be measured per worker process instead of for the whole system, and tests could be run on a dedicated (non-shared) machine to avoid inconsistent readings.